

1inch Aqua and Swap VM MVP v1.0 Security Audit

: 1inch Aqua and Swap VM MVP v1.0

Mar 27, 2026

Revision 1.1

ChainLight@Theori

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

Table of Contents

1inch Aqua and Swap VM MVP v1.0 Security Audit	1
Table of Contents	2
Executive Summary	4
Audit Overview	8
Scope	8
Code Revision	9
Severity Categories	10
Status Categories	11
Finding Breakdown by Severity	13
Findings	14
Summary	14
#1 1inch-AQUASWAPVM-001 PeggedSwap bidirectional pricing diverges with asymmetric x0/y0	17
#2 1inch-AQUASWAPVM-002 Static balances with decay can underflow on reverse swaps	19
#3 1inch-AQUASWAPVM-003 Aqua virtual balances break with fee-on-transfer or rebasing tokens	22
#4 1inch-AQUASWAPVM-004 Maker-controlled extruction target can constrain amounts within taker thresholds	24
#5 1inch-AQUASWAPVM-005 Backward jumps can trap the VM in OOG loops	27
#6 1inch-AQUASWAPVM-006 ExactOut feeBps==BPS causes division-by-zero	30
#7 1inch-AQUASWAPVM-007 Aqua mode + Balances opcodes can desync and revert	32
#8 1inch-AQUASWAPVM-008 XYCConcentrate path causes dynamic balance underflow	35
#9 1inch-AQUASWAPVM-009 Signature-mode orders ignore ship caps in Aqua router	37
#10 1inch-AQUASWAPVM-010 Aqua ExactIn pre-push reverts unless fee is pre-pushed under Aqua fee opcode	40
#11 1inch-AQUASWAPVM-011 Calldata slice underflow triggers OOG execution	42
#12 1inch-AQUASWAPVM-012 Aqua fee pulls desync XYCConcentrate liquidity scale	45
#13 1inch-AQUASWAPVM-013 Minor Suggestions	48
Revision History	50

Executive Summary

Beginning on January 12, 2026, ChainLight of Theori conducted a 3 week security audit of 1inch Aqua, swap-vm, solidity-utils's Solidity contracts. Aqua and SwapVM MVP v1.0 form a programmable on-chain trading stack. Aqua provides shared virtual liquidity balances, while SwapVM interprets maker-specified bytecode instructions, hooks, and traits to compute swap outcomes and enforce taker constraints. This assessment focused on the Aqua core contracts, SwapVM routers/opcodes/instructions, and supporting utilities in solidity-utils.

We identified **13 findings: Medium 4, Low 4, Informational 5**. The most material risks are order-level DoS and accounting mismatches driven by instruction combinations, router/mode selection, and edge-case parameterization.

Key themes across findings are **router/mode mismatches, VM execution safety issues** (unbounded loops, calldata slicing), **virtual-to-actual balance drift**, and **maker-controlled program risks** such as extruction-driven threshold capture.

Threat Model / Scenarios

- **Assets:** maker funds/approvals, Aqua internal balances, pool balances, fee accounting, price/quote correctness, and order liveness.
- **Adversaries:** malicious or careless makers/takers and untrusted external components (tokens, hooks/callbacks/extruction, fee providers, oracles).
- **Primary objectives:** keep balances correct, honor taker limits, ensure predictable execution, and prevent replay or reentrancy abuse.
- **Representative scenarios:**
 - replayable orders without invalidators
 - router/mode mismatches bypassing caps.
 - unbounded execution leading to gas exhaustion.
 - token-pair mixups when pairs are not enforced.
 - external dependencies causing gas blowups or reverts.
 - virtual-to-actual balance drift.
 - Aqua fee pulls causing liquidity scaling drift.
 - parameter mistakes that make orders unexecutable.

Trust Boundaries

- **Untrusted:** token behavior, maker/taker-provided program inputs, hooks/callbacks/extruction, fee providers/oracles, router/integrator inputs.

- **Untrusted (execution-level):** external call return data size/content, router selection/aggregation logic.
- **Trusted (as implemented):** in-scope contract logic, the order-level reentrancy guard, and signature/Aqua balance checks when inputs are well-formed.

Assumptions

- Aqua orders are reusable unless explicitly invalidated; signature orders can be replayed without invalidators.
- Router choice is a safety boundary; Aqua-mode orders should use the Aqua router.
- External dependencies are untrusted and must be vetted off-chain.
- Token compatibility is assumed unless explicitly restricted.
- Enforcing the correct token pair is the maker's responsibility.
- Quotes are indicative; state may change between quote and swap.
- Makers follow documented router/mode and instruction-combination constraints.

Boundaries and Constraints

- Instructions can be combined freely; some combinations create bad UX or unfavorable outcomes unless constrained.
- Using the general router with Aqua-mode orders can bypass intended Aqua-only constraints.
- Deployment, operations, and off-chain routing/integration are out of scope.
- Most risks are configuration-dependent and appear as order-level reverts or pricing inconsistencies.
- Chain-level economic attacks are out of scope.

Discussion

- **Instruction composability risk:** The instruction set is intentionally flexible, so some opcode orderings and parameter combinations are operationally invalid even if the bytecode is syntactically valid. The 1inch team response is to document which instructions can be composed, in what order, and to provide a builder contract for the common safe combinations rather than adding broad on-chain guardrails.
- **Hook surfaces and shared state:** Maker hooks remain an unconstrained extension surface and may share state across multiple orders, so cross-order sequencing and state-coupling risks remain part of the operating model. The 1inch team explicitly treats hook safety as a shared responsibility of maker and taker, with the expectation that both parties understand the hook behavior before use.
- **Router/mode boundaries:** Aqua mode exposed via the general SwapVM router remains a configuration boundary that depends on correct route selection and integrator discipline. The

1inch team prefers not to add extra checks that increase main-flow gas and instead assumes obviously bad strategies should remain unfilled rather than being proactively blocked on-chain.

- **Extraction and arbitrary external execution:** Extraction remains a maker-controlled arbitrary external execution surface. The 1inch team acknowledges that this extension cannot be made completely safe at the protocol level and expects takers to validate strategies that include Extraction before filling them.
- **Externalized dependencies:** Hooks, callbacks, extraction targets, fee providers, and oracles remain intentionally externalized trust boundaries. The 1inch team position is that complete user safety cannot be guaranteed at the protocol level for these dependencies, so off-chain participant validation and operator guidance are expected to carry much of the safety burden.
- **Fee-provider trust model:** Dynamic protocol fee providers are treated as trusted data sources in the intended operating model. The 1inch team does not plan to add additional returndata or gas-hardening checks that would increase the positive-path gas cost, and views a compromised fee provider as a broader dependency failure scenario rather than something to fully solve in-protocol.
- **Token compatibility policy:** Non-standard ERC20s such as fee-on-transfer or rebasing tokens can violate accounting assumptions. The 1inch team stated that v1 is intended for standard ERC-20 tokens only, and that support for other token classes would be considered separately.
- **Anti-spam economics:** If Aqua is permissionless for makers, dead or unfillable orders may still impose operational overhead on indexers or services. The 1inch team views this primarily as a chain-cost and service-ingestion issue rather than a direct contract-level vulnerability.
- **Maker UX and guidance gap:** Because makers can freely combine instructions, safe operation depends heavily on documentation-defined composition rules, trusted integrations, and parameter selection. The 1inch team stated that the README, whitepaper, and other documentation are being updated, and that a builder contract is planned to cover most allowed use cases and combinations.

Summary of Findings

The audit revealed a total **13** issues, categorized by severity as follows:

- **Medium:** 4 issues
- **Low:** 4 issues
- **Informational:** 5 issues

Audit Overview

Scope

Name	1inch Aqua and Swap VM MVP v1.0 Security Audit
Target / Version	The audit scope is as follows: • Aqua: https://github.com/1inch/aqua 89cedfa608aa9fdc60d8cd936b76a540e4b9346f ◦ src/*.sol ◦ src/libs/Balance.sol • SwapVM: https://github.com/1inch/swap-vm 48401a65362f8b1f47c39df9f74a287415da22d3 ◦ src/*.sol ◦ src/routers/AquaSwapVMRouter.sol ◦ src/opcodes/AquaOpcodes.sol ◦ src/instructions/Balances.sol ◦ src/instructions/Controls.sol ◦ src/instructions/Decay.sol ◦ src/instructions/Extraction.sol ◦ src/instructions/Fee.sol ◦ src/instructions/PeggedSwap.sol ◦ src/instructions/XYCConcentrate.sol ◦ src/instructions/XYCSwap.sol ◦ src/libs/MakerTraits.sol ◦ src/libs/PeggedSwapMath.sol ◦ src/libs/TakerTraits.sol ◦ src/libs/VM.sol • Solidity-utils: https://github.com/1inch/solidity-utils 38e5f8de705a8821237520ae06f3e9b0d40daf01 ◦ contracts/libraries/Calldata.sol ◦ contracts/libraries/CalldataPtr.sol ◦ contracts/libraries/Transient.sol ◦ contracts/libraries/TransientLock.sol ◦ contracts/mixins/Simulator.sol ◦ contracts/mixins/Multicall.sol

	Out of scope: • Components or services not listed above • External dependencies (oracles, fee providers, hooks/callbacks, tokens) were evaluated at interface boundaries only.
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

The final SwapVM patch commit is as follows:

- **SwapVM:** <https://github.com/1inch/swap-vm>
b2daef83656b1ccbfc1c83d3df869c0ebb60badc

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

Finding Breakdown by Severity

Category	Count	Findings
Critical	0	N/A
High	0	N/A
Medium	4	1inch-AQUASWAPVM-004 1inch-AQUASWAPVM-008 1inch-AQUASWAPVM-009 1inch-AQUASWAPVM-010
Low	4	1inch-AQUASWAPVM-001 1inch-AQUASWAPVM-003 1inch-AQUASWAPVM-005 1inch-AQUASWAPVM-011
Informational	5	1inch-AQUASWAPVM-002 1inch-AQUASWAPVM-006 1inch-AQUASWAPVM-007 1inch-AQUASWAPVM-012 1inch-AQUASWAPVM-013
Note	0	N/A

Findings

Summary

#	ID	Title	Severity	Status
1	1inch-AQUASWAPVM-001	PeggedSwap bidirectional pricing diverges with asymmetric x0/y0	Low	Patched
2	1inch-AQUASWAPVM-002	Static balances with decay can underflow on reverse swaps	Informational	Acknowledged
3	1inch-AQUASWAPVM-003	Aqua virtual balances break with fee-on-transfer or rebasing tokens	Low	Acknowledged
4	1inch-AQUASWAPVM-004	Maker-controlled extraction target can constrain amounts within taker thresholds	Medium	Acknowledged
5	1inch-AQUASWAPVM-005	Backward jumps can trap the VM in OOG loops	Low	Acknowledged
6	1inch-AQUASWAPVM-006	ExactOut feeBps==BPS causes division-by-zero	Informational	Acknowledged
7	1inch-AQUASWAPVM-007	Aqua mode + Balance opcodes can desync and revert	Informational	Acknowledged

#	ID	Title	Severity	Status
8	1inch-AQUASWAPVM-008	XYCConcentrate path causes dynamic balance underflow	Medium	Patched
9	1inch-AQUASWAPVM-009	Signature-mode orders ignore ship caps in Aqua router	Medium	Acknowledged
10	1inch-AQUASWAPVM-010	Aqua ExactIn pre-push reverts unless fee is pre-pushed under Aqua fee opcode	Medium	Acknowledged
11	1inch-AQUASWAPVM-011	Calldata slice underflow triggers OOG execution	Low	Acknowledged
12	1inch-AQUASWAPVM-012	Aqua fee pulls desync XYCConcentrate liquidity scale	Informational	Acknowledged
13	1inch-AQUASWAPVM-013	Minor Suggestions	Informational	Acknowledged

#1 1inch-AQUASWAPVM-001 PeggedSwap bidirectional pricing diverges with asymmetric x0/y0

ID	Summary	Severity
1inch-AQUASWAPVM-001	In <code>PeggedSwap._peggedSwapGrowPriceRange2D()</code> , <code>rateIn / rateOut</code> follow <code>tokenIn / tokenOut</code> ordering while <code>config.x0 / config.y0</code> stay fixed. If a maker sets <code>x0 != y0</code> and permits both directions, the same order applies different normalization bases when the direction flips. As a result, a taker can receive different quotes depending on swap direction for a single order.	Low

Description

When direction flips, `rateIn/rateOut` swaps, but `config.x0 / config.y0` do not, so the normalization denominators used by the invariant ($u = x \cdot \text{ONE} / \text{config.x0}$, $v = y \cdot \text{ONE} / \text{config.y0}$) can end up applied to the opposite token compared to the rate mapping. `PeggedSwap._peggedSwapGrowPriceRange2D()` uses `rateLt / rateGt` to normalize by `tokenIn < tokenOut`, but invariant and u / v normalization always reference fixed `config.x0` and `config.y0`. When `x0 != y0`, swapping `tokenIn` and `tokenOut` changes the applied rate but keeps the denominators fixed, yielding different pricing bases per direction. `orderHash` does not include `tokenIn` or `tokenOut`, and `SwapVM.swap()` / `SwapVM.quote()` take direction as an input. `MakerTraits.validate()` does not enforce a direction constraint by default. As a result, if a maker allows bidirectional execution with asymmetric `x0 / y0`, the order can quote different prices by direction, which is a configuration-dependent behavior.

Reproduction Scenario:

1. Maker builds an order with `useAquaInsteadOfSignature = true` and a program using `PeggedSwap._peggedSwapGrowPriceRange2D()` with asymmetric `x0 / y0`.
2. Maker ships Aqua balances for both tokens under `strategyHash = orderHash` (SwapVM uses `orderHash` as Aqua `strategyHash`).
3. Taker calls `SwapVM.quote()` in one direction and records the output.

4. Taker calls `SwapVM.quote()` in the reverse direction and observes a different output because `rateIn/rateOut` swaps, but `config.x0/y0` do not.

Impact

Low

Bidirectional swaps on orders with asymmetric normalization ($x_0 \neq y_0$) may result in direction-dependent pricing in some configurations. Makers expecting symmetric pricing in both directions could observe differing quotes. If $x_0 == y_0$, this direction-dependent effect largely disappears. This is configuration-dependent and occurs when a single order allows both directions.

Recommendation

If symmetric pricing is desired, consider (a) mapping x_0 / y_0 using the same token-ordering rule as `rateLt / rateGt` so the denominators swap when direction swaps, or (b) limiting swaps to a single direction when using asymmetric x_0 / y_0 . A simpler configuration mitigation is to keep $x_0 == y_0$ when bidirectional swaps are expected.

Remediation

Patched

It has been patched as recommended.

<https://github.com/1inch/swap-vm/pull/68>

#2 1inch-AQUASWAPVM-002 Static balances with decay can underflow on reverse swaps

ID	Summary	Severity
1inch-AQUASWAPVM-002	When a maker builds a repeatable order that combines <code>Balances._staticBalancesXD()</code> with <code>Decay._decayXD()</code> , the decay offsets grow across executions while the underlying balances remain fixed. A taker can then execute swaps in one direction until the stored offset exceeds <code>balanceOut</code> , causing a reverse-direction swap on the same order to underflow and revert. This makes bidirectional execution of such orders unreliable under those configurations.	Informational

Description

This path requires a program that combines `Balances._staticBalancesXD()`, `Decay._decayXD()`, and a swap instruction (for example, `XYCSwap._xycSwapXD()`) under the full `SwapVMRouter` opcode set. `AquaSwapVMRouter` does not include `Balances` opcodes, and `LimitOpcodes` does not include `Decay._decayXD()`, so this combination is limited to full-opcode orders. Offsets are stored per `orderHash`, token, and direction, and are applied before each swap by increasing `balanceIn` and decreasing `balanceOut`.

With static balances, repeated executions in one direction accumulate offsets that are later subtracted from a fixed `balanceOut` in the reverse direction. The underflow typically appears after a large swap in one direction followed by a smaller swap in the opposite direction. This does not occur for single-use orders with invalidators, `decayPeriod = 0`, or when dynamic balances are used.

Reproduction Scenario:

1. Maker builds a repeatable order (no invalidator) using `Balances._staticBalancesXD()` with balances of 1000e18/1000e18, `Decay._decayXD()` with `decayPeriod = 3600`, and `XYCSwap._xycSwapXD()` under `SwapVMRouter`.

2. Taker executes a large swap in one direction (for example, uses 1_000_000e18, tokenA → tokenB) to accumulate a large decay offset.
3. Taker executes the reverse-direction swap (tokenB → tokenA), which reverts with an arithmetic underflow once the decayed offset exceeds the static `balanceOut`.

Impact

Informational

The affected order may revert on reverse-direction swaps once the accumulated decay offset exceeds the static `balanceOut`. This is configuration-dependent and limited to repeatable orders that combine static balances with decay; it does not affect single-use orders or dynamic-balance paths.

Recommendation

Avoid using `Decay._decayXD()` together with `Balances._staticBalancesXD()`, or add a guard that prevents the decayed offset from exceeding `balanceOut` before subtraction. Prefer pairing decay with dynamic balances, which are designed to evolve with stateful execution. If static balances are used, consider requiring an invalidator to prevent repeated execution.

Remediation

Acknowledged

The 1inch team has acknowledged. Decay instruction with staticBalances is incorrect usage. Intentionally excluded from LimitOpcodes.

#3 1inch-AQUASWAPVM-003 Aqua virtual balances break with fee-on-transfer or rebasing tokens

ID	Summary	Severity
1inch-AQUASWAPVM-003	Because <code>Aqua.push()</code> / <code>Aqua.pull()</code> update <code>_balances</code> using nominal amounts without reconciling actual <code>balanceOf</code> changes, a maker who uses fee-on-transfer or rebasing tokens can cause virtual and actual balances to diverge. A taker swap can then revert or misprice when the contract tries to transfer amounts it does not actually hold, making swaps unreliable for non-standard tokens.	Low

Description

`Aqua.rawBalances()` and `Aqua.safeBalances()` return only internal storage values and never check actual token balances. In `Aqua.push()` and `Aqua.pull()`, the contract updates `_balances` before executing the token transfer, assuming standard ERC20 behavior. This means `_balances` can diverge from actual balances when the token does not transfer the exact nominal amount.

With `useAquaInsteadOfSignature = true` and `useTransferFromAndAquaPush = true`, the taker transfer for `tokenIn` is split into two transfers: taker → SwapVM, then `Aqua.push()` (SwapVM → maker). For fee-on-transfer `tokenIn`, SwapVM receives less than `amountIn`, but `Aqua.push()` still attempts to transfer the full nominal amount, so `safeTransferFrom()` can revert. For rebasing `tokenOut`, a negative rebase reduces the maker's actual balance while `_balances` remain unchanged, so `Aqua.pull()` later reverts even though virtual balances appear sufficient.

Reproduction Scenario:

1. Maker creates an Aqua order (`useAquaInsteadOfSignature = true`) and ships balances via `aqua.ship(address(swapVM), abi.encode(order), [tokenIn, tokenOut], [balanceIn, balanceOut])`, so `strategyHash = orderHash`.

2. Case A (fee-on-transfer `tokenIn`): taker swaps with `useTransferFromAndAquaPush = true`; the taker transfers to SwapVM, which is fee-charged, then `Aqua.push()` attempts to send the full nominal amount and reverts due to insufficient SwapVM balance.
3. Case B (rebasing `tokenOut`): a negative rebase reduces the maker's actual `tokenOut` balance; the next swap calls `Aqua.pull()` for the nominal `amountOut` and reverts because the actual balance is lower.

Impact

Low

When fee-on-transfer or rebasing tokens are used, Aqua's virtual balances may diverge from actual balances, potentially causing swaps to fail or behave unexpectedly relative to the maker's intended reserves. This is a compatibility issue with non-standard tokens and does not affect standard ERC20 behavior.

Recommendation

To reduce virtual-balance drift and transfer failures, the protocol could either:

1. **Track actual transferred amounts:** Compare the maker's `balanceOf` before and after each `push()` or `pull()` and update the virtual balances by the actual delta rather than the nominal amount. This helps keep `_balances` aligned with on-chain state.
2. **Restrict unsupported tokens:** Consider a ban or allowlist so that fee-on-transfer and rebasing tokens are not used with Aqua. By disallowing these non-standard ERC-20s, the virtual balance model stays closer to standard token semantics, reducing accounting mismatches.

Remediation

Acknowledged

The 1inch team has acknowledged. Fee-on-transfer tokens not supported in current version.

#4 1inch-AQUASWAPVM-004 Maker-controlled extruction target can constrain amounts within taker thresholds

ID	Summary	Severity
1inch-AQUASWAPVM-004	<code>Extruction._extruction()</code> lets a maker-controlled target overwrite <code>ctx.swap</code> after reading taker arguments, while <code>SwapVM.swap()</code> only fixes one side of the trade and treats the other as a threshold. As a result, the maker can reduce <code>amountOut</code> to <code>minOut</code> in <code>ExactIn</code> or increase <code>amountIn</code> up to <code>maxIn</code> in <code>ExactOut</code> , and the trade still passes taker validation. This enables execution at the worst acceptable price for the taker when extruction is used.	Medium

Description

`Extruction._extruction()` passes the current `SwapRegisters` and `ctx.takerArgs()` (taker `instructionsArgs`) to an external contract and unconditionally replaces `ctx.swap` with `updatedSwap`. `SwapVM.swap()` then runs `takerTraits.validate()`, which enforces `takerAmount == amountIn` for `ExactIn` or `takerAmount == amountOut` for `ExactOut`. The remaining side is only constrained by `minOut` or `maxIn`, allowing a maker-controlled target to adjust values within those thresholds.

If the taker encodes the threshold in `instructionsArgs`, a maker-controlled target can read it and clamp the output to the lowest acceptable amount. In `ExactIn`, `amountIn` is fixed, but `amountOut` can be reduced to `minOut`. In `ExactOut`, `amountOut` is fixed, but `amountIn` can be increased up to `maxIn`. This behavior stays within the taker's specified limits and represents a maker-controlled execution flexibility rather than an unrestricted exploit.

Reproduction Scenario:

1. Maker deploys an extruction target that reads `instructionsArgs` and clamps `amountOut` to a provided `minOut`, then includes it in a program around `XYCSwap._xycSwapXD()` (for example, `staticBalances` → `extruction` → `xycSwap` → `extruction`).

2. Taker submits an ExactIn swap with a `minOut` threshold encoded in both taker traits and `instructionsArgs`.
3. The extruction target sets `amountOut = minOut`, and `SwapVM.swap()` settles the trade at the worst acceptable price while still passing `takerTraits.validate()`.

Impact

Medium

Takers may receive execution at the edge of their own threshold constraints when an order includes `Extruction._extruction()`. This is a maker-controlled outcome within taker-defined limits and can be viewed as a UX consideration for orders that opt into extruction behavior.

Recommendation

Document this as a maker-controlled execution feature. Consider explicit taker opt-in or a clear warning when an order includes `Extruction._extruction()` that can rewrite `amountIn/amountOut`. UI should ideally surface this capability so takers understand their thresholds can be clamped by a maker-controlled target.

Remediation

Acknowledged

The 1inch team has acknowledged. Extruction extension inherently dangerous. Cannot be made fully secure. Takers must validate strategies before use.

#5 1inch-AQUASWAPVM-005 Backward jumps can trap the VM in

OOG loops

ID	Summary	Severity
1inch-AQUASWAPVM-005	Because the maker can set <code>nextPC</code> to a lower offset without any bounds or step limits, a crafted program can force <code>ContextLib.runLoop()</code> to loop indefinitely, causing taker calls to revert with out-of-gas at the order level.	Low

Description

The intended use of jump opcodes is forward-only control flow to skip blocks or select a path. However, `_jump()` and the conditional jump opcodes accept any `nextPC` and store it in `ctx.vm.nextPC` without verifying `nextPC >= currentPC` or checking program bounds. `ContextLib.runLoop()` does not enforce a maximum number of steps and continues executing while updating `pc` from `ctx.vm.nextPC`.

If a maker miscalculates offsets or intentionally embeds a backward jump, the VM can re-enter earlier instructions indefinitely. This causes `SwapVM.quote()` and `SwapVM.swap()` to run until gas exhaustion. The behavior is deterministic for any program containing a backward jump.

Reproduction Scenario:

1. A maker builds a program that contains only `Controls._jump()` with `nextPC = 0`, so execution always jumps back to the first instruction.
2. A taker calls `SwapVM.quote()` for the order with a bounded gas limit.
3. `ContextLib.runLoop()` repeatedly executes the same jump without a step limit until out-of-gas and the call reverts.

Impact

Low

Orders that contain a backward jump may be unexecutable and can lead to failed calls with gas spent by takers. The impact is limited to the specific order and does not imply protocol-wide denial

of service or direct fund loss.

Recommendation

Consider requiring `nextPC >= currentPC` in `Controls._jump()`, `Controls._jumpIfTokenIn()`, and `Controls._jumpIfTokenOut()`. Alternatively, consider adding a maximum step counter in `ContextLib.runLoop()` and reverting when a limit is exceeded. If bounded loops are a desired feature, a dedicated loop opcode with explicit iteration limits could be introduced.

Remediation

Acknowledged

The 1inch team has acknowledged. Execution cost concerns across all cases. Takers must validate strategies before use.

#6 1inch-AQUASWAPVM-006 ExactOut feeBps==BPS causes division-by-zero

ID	Summary	Severity
1inch-AQUASWAPVM-006	Because <code>Fee._feeAmountIn()</code> divides by <code>BPS - feeBps</code> and the builder allows <code>feeBps == BPS</code> , a maker can configure ExactOut orders that always revert, causing taker calls to fail with a deterministic div-by-zero.	Informational

Description

In ExactOut mode, `Fee._feeAmountIn()` divides by `BPS - feeBps`. If `feeBps` is set to `BPS`, the division by zero reverts on every ExactOut execution. ExactIn does not use this formula, so the failure is isolated to ExactOut flows.

`FeeArgsBuilder.buildFlatFee()` validates `feeBps <= BPS`, which allows a misconfigured order to pass creation. As a result, a maker can unintentionally create an order that always fails for ExactOut swaps. This is a deterministic configuration failure limited to ExactOut usage.

Reproduction Scenario:

1. A maker builds a program with `Fee._flatFeeAmountInXD()` using `feeBps == BPS`, followed by `XYCSwap._xycSwapXD()`.
2. A taker calls `SwapVM.quote()` in ExactOut mode for that order.
3. `Fee._feeAmountIn()` computes `feeAmountIn = amountIn * feeBps / (BPS - feeBps)`, the denominator becomes zero, and the call reverts.

Impact

Informational

Orders with `feeBps == BPS` cannot be executed in ExactOut mode and may lead to failed calls with gas spent by takers. Funds are not moved, and the impact is limited to this specific configuration.

Recommendation

Consider requiring `feeBps < BPS` in `FeeArgsBuilder.buildFlatFee()` or adding an explicit check in `Fee._feeAmountIn()` that rejects `feeBps == BPS` with a clear error. If 100% fees must be representable, consider documenting `ExactOut` as unsupported for that configuration.

Remediation

Acknowledged

The 1inch team has acknowledged. Lower fee orders should be treated as invalid by takers. Adding weaker conditions increases gas costs for all orders. Takers must validate strategies before use.

#7 1inch-AQUASWAPVM-007 Aqua mode + Balances opcodes can desync and revert

ID	Summary	Severity
1inch-AQUASWAPVM-007	Because a maker can run Aqua-mode orders through <code>SwapVMRouter</code> and then invoke <code>Balances</code> opcodes that overwrite or contradict preloaded <code>Aqua.safeBalances()</code> , the order can either revert immediately or price against different balances than settlement, causing order-level failures or inconsistent execution.	Informational

Description

In Aqua mode (`useAquaInsteadOfSignature = true`), `SwapVM.swap()` and `SwapVM.quote()` pre-load `ctx.swap.balanceIn/out` via `Aqua.safeBalances()` before executing the program. `SwapVMRouter` exposes the full opcode set, including `Balances`, so Aqua orders can include `Balances._staticBalancesXD()` or `Balances._dynamicBalancesXD()` even though `AquaSwapVMRouter` cannot.

Static Balance Conflict: `_staticBalancesXD()` requires `ctx.swap.balanceIn == 0` and `ctx.swap.balanceOut == 0`. This means any Aqua-mode path that reaches this opcode will revert when non-zero Aqua balances have been shipped, making the path unexecutable in that configuration.

Reproduction Scenario:

1. Maker creates a `SwapVMRouter` order with `useAquaInsteadOfSignature = true` and a program that includes `Balances._staticBalancesXD()`, then ships non-zero Aqua balances.
2. Taker calls `SwapVM.quote()` or `SwapVM.swap()` and execution reaches `_staticBalancesXD()` after Aqua preload.
3. `_staticBalancesXD()` reverts because the balances are already non-zero.

Dynamic Balance Conflict: `_dynamicBalancesXD()` loads or initializes balances from its own storage and overwrites the preloaded Aqua balances in `ctx.swap.balanceIn/out`. Swap

pricing is then computed against the dynamic balances, but settlement still uses Aqua balances. If the stored dynamic balances do not match the shipped Aqua balances, the swap can revert (insufficient Aqua balance) or execute with pricing that does not reflect the shipped Aqua balances.

Reproduction Scenario (Dynamic):

1. Maker creates a `SwapVMRouter` order with `useAquaInsteadOfSignature = true` and a program that includes `Balances._dynamicBalancesXD()`, then ships non-zero Aqua balances.
2. Taker calls `SwapVM.swap()` and execution reaches `_dynamicBalancesXD()` after Aqua preload.
3. `_dynamicBalancesXD()` overwrites `ctx.swap.balanceIn/out` with dynamic balances; the swap is priced against these values while settlement uses Aqua balances, producing a revert or pricing that does not align with shipped Aqua balances when the two sources diverge.

Impact

Informational

Static balances in Aqua mode may revert when non-zero Aqua balances are shipped, which can make the affected path unusable and may result in failed calls with gas spent. Dynamic balances may also lead to reverts or pricing that diverges from shipped Aqua balances when the two sources differ. This is configuration-dependent and affects expected execution semantics rather than implying direct loss.

Recommendation

Consider isolating Aqua mode from `Balances` opcodes in `SwapVMRouter`, for example by:

1. Enforcing that `useAquaInsteadOfSignature = true` orders must use `AquaSwapVMRouter`, or
2. Adding a guard that rejects `Balances` opcodes when Aqua mode is enabled.

Remediation

Acknowledged

The 1inch team has acknowledged. `Balances` usage in Aqua is considered incorrect. `static/dynamicBalances` opcodes intentionally excluded from `AquaOpcodes`.

#8 1inch-AQUASWAPVM-008 XYCConcentrate path causes dynamic balance underflow

ID	Summary	Severity
1inch-AQUASWAPVM-008	Because a maker can combine XYCConcentrate with dynamic balances to inflate in-memory balanceOut without updating storage, ExactIn swaps can compute amountOut above the stored balance and then underflow on settlement, so takers may get a quote that later reverts on swap.	Medium

Description

Balances._dynamicBalancesXD() loads ctx.swap.balanceOut from storage using Balances._loadBalances(). XYCConcentrate._xycConcentrateGrowLiquidity2D() then overwrites ctx.swap.balanceOut in memory with XYCConcentrate.concentratedBalance(). When liquidity[orderHash] == 0 and deltaOut > 0, concentratedBalance() returns balance + deltaOut, which inflates the in-memory balance.

XYCSwap._xycSwapXD() computes amountOut using the inflated memory balance. Balances._dynamicBalancesXD() later subtracts swapAmountOut from the stored balances[orderHash][tokenOut], so an underflow occurs when swapAmountOut exceeds the stored balance. In ExactIn, a sufficiently large amountIn makes amountOut approach the inflated concentratedBalance() result, which makes the underflow reachable. SwapVM.quote() is a static context and does not update storage, which explains why a quote can succeed while the corresponding swap reverts.

Reproduction Scenario:

1. A maker builds an order that runs Balances._dynamicBalancesXD() → XYCConcentrate._xycConcentrateGrowLiquidity2D() → XYCSwap._xycSwapXD() with deltaOut > 0 and no existing liquidity (liquidity[orderHash] == 0).
2. A taker requests an ExactIn quote with a large amountIn, producing an amountOut based on the inflated in-memory balance.

3. The swap with the same parameters reverts when `Balances._dynamicBalancesXD()` subtracts `swapAmountOut` from storage and underflows.

Impact

Medium

`SwapVM.quote()` may return a result that `SwapVM.swap()` cannot execute for the specific opcode sequence and parameter set, leading to quote/swap inconsistency and potential failed calls with gas spent for affected orders. No direct fund loss or protocol-wide impact is expected.

Recommendation

Consider adding a guard in `Balances._dynamicBalancesXD()` that enforces `swapAmountOut <= balances[orderHash][tokenOut]` before subtraction. Alternatively, consider capping the computed output in `XYCSwap._xycSwapXD()` to the stored balance for dynamic-balance paths.

Remediation

Patched

The 1inch team has patched this issue. The issue was based on the previous ConcentrateXYC design, specifically the first-fill branch in `concentratedBalance()` when `liquidity[orderHash] == 0`. The current ConcentrateXYC implementation no longer contains `liquidity[orderHash]`, `concentratedBalance()`, or `_updateScales()`, so the specific stateful mechanism described in the original issue no longer applies as written. In addition, comments were added to related instructions, including `Balances`, documenting quote/swap inconsistency.

<https://github.com/1inch/swap-vm/pull/82>

#9 1inch-AQUASWAPVM-009 Signature-mode orders ignore ship caps in Aqua router

ID	Summary	Severity
1inch-AQUASWAPVM-009	Because a maker can submit <code>useAquaInsteadOfSignature = false</code> orders through <code>AquaSwapVMRouter</code> , the router skips <code>Aqua.safeBalances()</code> and settles via <code>safeTransferFrom</code> , so shipped Aqua balances do not cap execution and takers may drain wallet balances beyond the shipped amount when allowances permit.	Medium

Description

In `SwapVM.swap()`, the `useAquaInsteadOfSignature` branch either loads `Aqua.safeBalances()` or performs signature verification. When `useAquaInsteadOfSignature = false`, `ctx.swap.balanceIn/out` starts at zero, and no Aqua balances are loaded. The settlement path calls `_transferOrPull()` which selects `IERC20.safeTransferFrom()` when `useAqua` is false, so `Aqua.pull()` is never used and `Aqua.ship()` caps are not applied.

Instruction sets like `XYCConcentrate._xycConcentrateGrowLiquidity2D()` can derive balances from deltas even when the in-memory balances are zero.

`XYCConcentrate.concentratedBalance()` returns `balance + delta` when `currentLiquidity == 0`, allowing `XYCSwap._xycSwapXD()` to compute a swap without Aqua balances. `Aqua.ship()` itself only stores virtual balances using `Balance.store()` and does not move tokens, so the maker wallet is the actual source of funds in signature mode.

Reproduction Scenario:

1. A maker ships Aqua balances but creates an `AquaSwapVMRouter` order with `useAquaInsteadOfSignature = false` and an opcode path that can compute from deltas, such as `XYCConcentrate._xycConcentrateGrowLiquidity2D()` → `XYCSwap._xycSwapXD()`.

2. A taker executes `SwapVM.swap()` and the program derives liquidity even though Aqua balances were not loaded.
3. Settlement pulls from the maker wallet via `safeTransferFrom`, leaving Aqua balances unchanged and allowing execution beyond the shipped cap when wallet balance and allowance are sufficient.

Impact

Medium

Shipped Aqua balances do not cap execution for signature-mode orders (`useAquaInsteadOfSignature = false`). If makers expect ship caps to apply but accidentally use signature mode, swaps can draw from the maker wallet beyond shipped balances when allowance and balances exist. If balances or allowances are insufficient, `IERC20.safeTransferFrom()` reverts and takers pay gas for failed swaps. The behavior is configuration-dependent and reflects a mode selection mismatch rather than a protocol-level bypass.

Recommendation

Reject `useAquaInsteadOfSignature = false` orders in `AquaSwapVMRouter`, or disallow opcode combinations (such as `XYCConcentrate`) that allow swaps to compute without Aqua balances in that path. Alternatively, enforce that Aqua routing always uses `Aqua.safeBalances()` and `Aqua.pull()` so ship caps are applied.

Remediation

Acknowledged

The 1inch team has acknowledged. Takers must validate strategies before use.

#10 1inch-AQUASWAPVM-010 Aqua ExactIn pre-push reverts unless fee is pre-pushed under Aqua fee opcode

ID	Summary	Severity
1inch-AQUASWAPVM-010	Because the maker's Aqua-mode program can pull fees during the run loop while <code>_transferIn()</code> still validates against the pre-fee <code>amountIn</code> , takers who pre-push without including the fee will fail validation and see their ExactIn swaps revert.	Medium

Description

This behavior occurs when `useAquaInsteadOfSignature = true`, the taker uses pre-push, and the program includes `Fee._aquaProtocolFeeAmountInXD()` or `Fee._aquaDynamicProtocolFeeAmountInXD()`. `SwapVM.swap()` snapshots `originalAquaBalanceIn` before `ContextLib.runLoop()`, and the fee opcodes call `Aqua.pull()` during the run loop. In ExactIn, `Fee._feeAmountIn()` temporarily reduces `ctx.swap.amountIn` for computation and then restores it.

The subsequent `SwapVM._transferIn()` check compares `Aqua.rawBalances()` against `originalAquaBalanceIn + amountIn`. Because the fee was already pulled during `ContextLib.runLoop()`, the check effectively requires an extra pre-push equal to the fee. As a result, takers must include the fee in pre-push (i.e., `prePush >= amountIn + fee`), or the swap reverts even though the fee has already been applied to pricing.

Reproduction Scenario:

1. A maker creates an Aqua-mode order with `useAquaInsteadOfSignature = true`, pre-push enabled (`useTransferFromAndAquaPush = false`), and a fee opcode such as `Fee._aquaProtocolFeeAmountInXD()`.
2. A taker pre-pushes exactly `amountIn` and calls `SwapVM.swap()` in ExactIn mode.
3. The fee is pulled during `ContextLib.runLoop()`, but `_transferIn()` still validates against `amountIn`, so the swap reverts unless the taker pre-pushes `amountIn + fee`.

Impact

Medium

ExactIn swaps may revert unless pre-push includes the fee when Aqua fee opcodes are used with pre-push. In practice, this means the taker may need to pre-push `amountIn + fee` to pass validation for this mode combination. The issue does not enable fund theft, but it can disrupt expected fee handling for this specific path.

Recommendation

Consider adjusting pre-push validation to account for fee pulls in ExactIn by validating against the net balance delta after `ContextLib.runLoop()`. Alternatively, consider disallowing Aqua fee opcodes when pre-push is used, or require explicit taker opt-in for this behavior.

Remediation

Acknowledged

The 1inch team has acknowledged this issue and added documentation describing the limitation.

#11 1inch-AQUASWAPVM-011 Calldata slice underflow triggers OOG execution

ID	Summary	Severity
1inch-AQUASWAPVM-011	Because makers can set non-monotonic slice indexes so <code>begin > end</code> and <code>Calldata.slice()</code> does not enforce ordering, the computed length underflows and <code>ContextLib.runLoop()</code> iterates over an oversized program, causing order-level OOG reverts for takers who execute the order.	Low

Description

Maker and taker slice offsets are encoded as bitfields, but no monotonicity checks are enforced in `MakerTraits._getDataSlice()` or `TakerTraits._getDataSlice()`. The `Calldata.slice(begin, end, exception)` overload only validates that `end` does not exceed `calls.length`; it does not enforce `begin <= end`. As a result, when `begin > end`, the length underflows to a huge value. For example, if `order.data` has length 2 (`0x1700`) and indices are set so `start = 0xFFFF` and `end = 0x0002`, then `res.length = 0x0002 - 0xFFFF = 2^256 - 65533`.

`SwapVM.swap()` and `SwapVM.quote()` use the resulting slices as `programPtr` and `takerArgsPtr`, so the VM operates on the malformed program. `ContextLib.runLoop()` uses `programBytes.length` as its upper bound and has no step limit, so execution consumes all gas. This is an order-level OOG failure, not a protocol-wide DoS. This behavior affects both signature and Aqua execution modes.

Reproduction Scenario:

1. A maker crafts an order with non-monotonic calldata slice offsets so the computed `begin` is greater than `end`, for example, by setting a large start index while keeping `order.data` very short.
2. A taker calls `SwapVM.swap()` or `SwapVM.quote()` for that order.
3. `Calldata.slice()` underflows the slice length, `ContextLib.runLoop()` iterates over the oversized program until gas is exhausted, and the call reverts.

Impact

Low

Malicious or misconfigured orders can revert with out-of-gas, causing taker gas loss and order-level execution failure. The behavior is deterministic once offsets are malformed. It affects both signature and Aqua execution modes but does not cause protocol-wide denial of service or direct fund loss.

Recommendation

Consider enforcing `begin <= end` when slicing in `MakerTraits._getDataSlice()` and `TakerTraits._getDataSlice()`, or adding this check inside `Calldata.slice()` before computing the length. This helps prevent oversized slices from reaching `ContextLib.runLoop()`.

Remediation

Acknowledged

The 1inch team has acknowledged. Gas optimization prioritized. No additional checks planned.

#12 1inch-AQUASWAPVM-012 Aqua fee pulls desync

XYCConcentrate liquidity scale

ID	Summary	Severity
1inch-AQUASWAPVM-012	Because XYCConcentrate updates liquidity after runLoop() without accounting for Aqua fee pulls, a maker who combines Aqua-fee opcodes with XYCConcentrate can leave the stored scale inconsistent with fee-adjusted reserves, leading to unexpected pricing for subsequent swaps.	Informational

Description

In Aqua mode (`useAquaInsteadOfSignature = true`), balances are maintained in Aqua, and fee opcodes such as `Fee._aquaProtocolFeeAmountInXD` pull fees directly from Aqua balances. `XYCConcentrate._xycConcentrateGrowLiquidityXD/2D()` calls `ctx.runLoop()` and then computes `liquidity[orderHash] = sqrt((balanceIn + amountIn) * (balanceOut - amountOut))`, but it does not account for the extra Aqua pulls that occurred during the `runLoop`. This means `liquidity[orderHash]` is updated using a balance state that ignores fee-induced reductions in Aqua reserves.

Because `concentratedBalance()` scales by `liquidity[orderHash]`, subsequent swaps use a scale that is offset from the fee-aware value and effectively loosen the intended concentration. This persists across swaps until liquidity is updated again. The combination is not blocked by the opcode set, so a maker can assemble it even though XYCConcentrate does not reconcile Aqua fee pulls by design.

In current XYCConcentrate test setups, the fee path is typically `Fee._flatFeeAmountInXD` with `useAquaInsteadOfSignature = false`, so Aqua fee pulls are not exercised in those combinations.

Reproduction Scenario:

1. Maker builds an Aqua order (`useAquaInsteadOfSignature = true`) and ships Aqua balances for `tokenIn` and `tokenOut`.

2. Program sequence includes `XYCConcentrate._xycConcentrateGrowLiquidity2D`, then an Aqua fee opcode (for example, `Fee._aquaProtocolFeeAmountInXD`), followed by `XYCSwap._xycSwapXD` in the same runLoop.
3. Taker executes a swap; Aqua pulls the fee during the runLoop, and `_updateScales` stores liquidity as if the fee had not reduced the reserves.

Impact

Informational

Using an unsupported Aqua-fee + XYCConcentrate combination may leave the liquidity scale out of sync with fee-adjusted reserves, resulting in a looser price curve than a maker expects. This is configuration dependent and requires Aqua mode plus Aqua fee opcodes executed within the XYCConcentrate runLoop. Funds are not directly lost, but pricing and risk assumptions for the strategy may be affected.

Recommendation

Consider documenting that XYCConcentrate does not reconcile Aqua fee pulls and that combining it with Aqua fee opcodes (for example, `_aquaProtocolFeeAmountOutXD`, `_aquaProtocolFeeAmountInXD`) is unsupported. Consider surfacing this as an explicit maker-facing warning in strategy builders and docs, so makers avoid assembling these open but unsafe combinations.

Remediation

Acknowledged

The 1inch team has acknowledged this issue and added documentation describing the limitation.

#13 1inch-AQUASWAPVM-013 Minor Suggestions

ID	Summary	Severity
1inch-AQUASWAPVM-013	The description includes multiple suggestions for preventing incorrect settings caused by operational mistakes, mitigating potential issues, improving code maturity and readability, and other minor issues.	Informational

Description

Operational Risk Mitigation / Sanity Check

1. In `XYCConcentrate`, `concentratedBalance()` computes `delta * currentLiquidity / initialLiquidity` when `currentLiquidity != 0`, but there is no `initialLiquidity > 0` check. To avoid division by zero, add a guard after parsing in `_xycConcentrateGrowLiquidity2D()` / `_xycConcentrateGrowLiquidityXD()` that enforces `initialLiquidity > 0`.
2. `Aqua.ship()` indexes `amounts[i]` without validating `tokens.length == amounts.length`, so a length mismatch can revert. Add `require(tokens.length == amounts.length)`.
3. `Decay._decayXD()` with `period = 0` effectively disables decay, so a hard security guard is not required. If desired, consider a policy-level constraint (configuration sanity) that requires `period > 0`.
4. In `_xycConcentrate*` (XD path), if `tokenIn / tokenOut` addresses are duplicated in the `tokens` list, `concentratedBalance()` can be applied multiple times to the same token. Because the updated `ctx.swap.balanceIn/out` is fed back into subsequent applications, liquidity can be inflated, and `_updateScales` becomes inconsistent with the intended pool state. Add a duplicate-token check when building or parsing the arguments.
5. In `Controls._onlyTakerTokenSupplyShareGte()`, the direct multiplications `balance * 1e18` and `minShareE18 * totalSupply` can overflow and cause unintended reverts. To avoid unnecessary execution failures even with well-behaved tokens, consider using `Math.mulDiv` (OpenZeppelin `Math.sol`) or a rearranged comparison that avoids intermediate overflow.

6. In `Aqua.dock()`, if `tokens.length == 0` and the strategy is uninitialized (`tokensCount == 0`), the function emits a `Docked` event without changing state. Consider adding `require(tokens.length > 0)` (or `tokensCount > 0`) to prevent spurious `Docked` events for empty inputs.

Impact

Informational

Recommendation

Consider applying the suggestions in the description above.

Remediation

Acknowledged

* The 1inch team has patched the XYC-related issues through a redesign of ConcentrateXYC. The previous stateful logic and old XD path described in the original issues no longer exist in the current implementation, so those issue mechanisms no longer apply as written. (Issues: 1, 4).

* The hardening issues in points 2, 3, 5, and 6 were acknowledged by the 1inch team.

Revision History

Version	Date	Description
1.0	Mar 6, 2026	Initial version
1.1	Mar 27, 2026	Final version

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

